Développement d'un site web dynamique (UAA12)

Exercices : exploitation de base de données

Objets d'apprentissage :

- ✔ Elaborer un formulaire sur la base d'une structure donnée.
- ✔ Construire une page WEB dynamique à l'aide du langage PHP.
- ✔ Se connecter à une base de données relationnelle à l'aide du langage PHP.
- ✔ Exécuter des requêtes de lecture et d'écriture à une base de données relationnelle.
- ✔ Concevoir un formulaire répondant aux exigences d'un cahier des charges.
- ✔ Se protéger des attaques WEB les plus courantes.
- ✔ Dynamiser un site WEB intégrant l'utilisation d'une base de données relationnelle.
- ✔ Vérifier la présence de codes malveillants dans les données reçues.
- ✔ Associer des balises HTML de formulaires à leur sémantique.
- ✔ Identifier des modèles et des bibliothèques provenant de tierces parties.
- ✔ Formuler la syntaxe des requêtes à une base de données relationnelle (SQL).
- ✔ Caractériser les attaques WEB les plus courantes.

Exercice 1 : le marché artisanal

L'objectif de cet exercice est de t'initier à la connexion PHP/MySQL et à l'affichage de données simples.

Etape 1

Dans phpMyAdmin, crée une base de données nommée `marche` puis exécute le script SQL suivant :

```sql
CREATE TABLE producteurs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    prenom VARCHAR(100),
    specialite VARCHAR(100),
    commune VARCHAR(100),
    annee_installation INT
);
INSERT INTO producteurs VALUES
(NULL, 'Dubois', 'Marie', 'Confitures', 'Liège', 2015),
(NULL, 'Renard', 'Paul', 'Fromages', 'Namur', 2009),
(NULL, 'Simon', 'Claire', 'Miel', 'Arlon', 2018),
(NULL, 'Lecomte', 'Jacques', 'Légumes bio', 'Dinant', 2012),
(NULL, 'Bastin', 'Sophie', 'Savons artisanaux', 'Huy', 2020);
```

Note. Comme tu peux le voir, il n'est pas nécessaire de spécifier l'identifiant (`id`) du producteur car c'est le SGBD qui s'en charge (clause `AUTO_INCREMENT`).

Etape 2

Crée un fichier `marche.php` et établis la connexion à la base de données à l'aide du code suivant :

```php
<?php
$conn = mysqli_connect('localhost', 'root', '', 'marche');
if (!$conn) {
   die('Erreur de connexion : ' . mysqli_connect_error());
}
?>
```

Vérifie que la connexion fonctionne (aucun message d'erreur ne doit s'afficher).

Etape 3

Ajoute le code PHP permettant d'exécuter la requête suivante et d'afficher les résultats dans un tableau HTML :

```php
<?php
$sql = "SELECT * FROM producteurs";
$result = mysqli_query($conn, $sql);
while ($row = mysqli_fetch_assoc($result)) {
    // à compléter
}
?>
```

Le tableau HTML doit comporter les colonnes suivantes : Nom, Prénom, Spécialité, Commune et Année d'installation.

Le code complété se trouve à la page suivante. Essaie d'abord de le faire toi-même avant d'aller voir !

Etape 4

Voici le code complété :

```php
<?php
$sql = "SELECT * FROM producteurs";
$result = mysqli_query($conn, $sql);
if ($result) {
    echo "<table>";
    echo "<tr>
            <th>Nom</th>
            <th>Prénom</th>
            <th>Spécialité</th>
            <th>Commune</th>
            <th>Année d'installation</th>
          </tr>";
    while ($row = mysqli_fetch_assoc($result)) {
        echo "<tr>";
        echo "<td>" . $row['nom'] . "</td>";
        echo "<td>" . $row['prenom'] . "</td>";
        echo "<td>" . $row['specialite'] . "</td>";
        echo "<td>" . $row['commune'] . "</td>";
        echo "<td>" . $row['annee_installation'] . "</td>";
        echo "</tr>";
    }
    echo "</table>";
} else {
    echo "Erreur lors de l'exécution de la requête : " .
mysqli_error($conn);
}
?>
```

Ajoute les règles CSS suivantes :

- Le tableau doit avoir une bordure grise et les cellules doivent être fusionnées (`border-collapse`)
- Une ligne sur deux doit avoir un fond légèrement coloré (utilise `:nth-child`)
- L'en-tête du tableau doit avoir un fond vert foncé avec du texte blanc

Etape 5

Modifie ta requête SQL pour n'afficher que les producteurs installés après 2014, triés par ordre alphabétique sur le nom de famille.

Le code complété se trouve à la page suivante. Essaie d'abord de le faire toi-même avant d'aller voir !

Etape 6

Voici le code complet :

```
<!DOCTYPE html>
<html>
<head>
    <title>Marché artisanal</title>
    <style>
        table {
            border: 2px solid grey;
            border-collapse: collapse;
            width: 100%;
        }
        th, td {
            border: 1px solid grey;
            padding: 6px;
        }
        th {
            background-color: darkgreen;
            color: white;
        }
        tbody tr:nth-child(even) {
            background-color: #f0f0f0;
        }
    </style>
</head>
<body>
<?php
$conn = mysqli_connect('localhost', 'root', '', 'marche');
if (!$conn) {
    die('Erreur de connexion : ' . mysqli_connect_error());
}
$sql = "SELECT * FROM producteurs WHERE annee_installation > 2014 ORDER BY
nom ASC";
$result = mysqli_query($conn, $sql);
```

```php
if ($result) {
    echo "<table>";
    echo "<thead>";
    echo "<tr>
            <th>Nom</th>
            <th>Prénom</th>
            <th>Spécialité</th>
            <th>Commune</th>
            <th>Année d'installation</th>
          </tr>";
    echo "</thead>";
    echo "<tbody>";
    while ($row = mysqli_fetch_assoc($result)) {
        echo "<tr>";
        echo "<td>" . $row['nom'] . "</td>";
        echo "<td>" . $row['prenom'] . "</td>";
        echo "<td>" . $row['specialite'] . "</td>";
        echo "<td>" . $row['commune'] . "</td>";
        echo "<td>" . $row['annee_installation'] . "</td>";
        echo "</tr>";
    }
    echo "</tbody>";
    echo "</table>";
} else {
    echo "Erreur lors de l'exécution de la requête : " .
mysqli_error($conn);
}
mysqli_close($conn);
?>
</body>
</html>
```

Exercice 2 : la fête foraine

L'objectif de cet exercice est d'apprendre à filtrer dynamiquement les données affichées selon des paramètres transmis via POST.

Etape 1

Dans phpMyAdmin, crée une base de données nommée `foraine` puis exécute le script SQL suivant :

```sql
CREATE TABLE attractions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    type VARCHAR(50),
    age_minimum INT,
    prix_ticket DECIMAL(4,2),
    places_par_tour INT
);

INSERT INTO attractions VALUES
(NULL, 'Grand Huit', 'Montagne russe', 12, 2.50, 20),
(NULL, 'Carrousel enchanté', 'Manège', 3, 1.00, 30),
(NULL, 'Chute libre', 'Sensations fortes', 14, 3.00, 8),
(NULL, 'Chambre des horreurs', 'Attraction', 10, 2.00, 15),
(NULL, 'Train fantôme', 'Attraction', 8, 1.50, 12),
(NULL, 'Chenille', 'Manège', 5, 1.00, 24),
(NULL, 'Tour de chute', 'Sensations fortes', 12, 2.50, 16);
```

Etape 2

Crée un fichier `foraine.php` qui affiche l'ensemble des attractions dans un tableau HTML avec toutes leurs informations.

Etape 3

Crée une page `choix.html` proposant un formulaire avec une liste déroulante permettant de choisir un type d'attraction (« Manège », « Montagne russe », « Sensations fortes », « Attraction »). Cette liste doit également proposer une option « Toutes les attractions » (valeur envoyée : une chaine de caractère vide). Le formulaire redirige vers `foraine.php` via la méthode POST.

Etape 4

Dans `foraine.php`, adapte ta requête SQL pour qu'elle ne retourne que les attractions du type choisi, si un paramètre POST est fourni. Si aucun paramètre n'est fourni, toutes les attractions sont affichées.

N'oublie pas d'utiliser les requêtes préparées (`mysqli_prepare()`) !

Le code complété se trouve à la page suivante. Essaie d'abord de le faire toi-même avant d'aller voir !

Etape 5

Voici le code complet de `foraine.php` :

```
<!DOCTYPE html>
<html>
<head>
    <title>Fête foraine</title>
</head>
<body>
<?php
$conn = mysqli_connect('localhost', 'root', '', 'foraine');
if (!$conn) {
    die('Erreur de connexion : ' . mysqli_connect_error());
}
if (isset($_POST['type']) && !empty($_POST['type'])) {
    $stmt = mysqli_prepare($conn, "SELECT * FROM attractions WHERE type
= ?");
    mysqli_stmt_bind_param($stmt, "s", $_POST['type']);
    mysqli_stmt_execute($stmt);
    $result = mysqli_stmt_get_result($stmt);
} else {
    $result = mysqli_query($conn, "SELECT * FROM attractions");
}
if ($result) {
    echo "<table>";
    echo "<thead>";
    echo "<tr>
            <th>Nom</th>
            <th>Type</th>
            <th>Âge minimum</th>
            <th>Prix (ticket)</th>
            <th>Places par tour</th>
          </tr>";
```

```
    echo "</thead>";
    echo "<tbody>";
    while ($row = mysqli_fetch_assoc($result)) {
        echo "<tr>";
        echo "<td>" . $row['nom'] . "</td>";
        echo "<td>" . $row['type'] . "</td>";
        echo "<td>" . $row['age_minimum'] . " ans</td>";
        echo "<td>" . $row['prix_ticket'] . " €</td>";
        echo "<td>" . $row['places_par_tour'] . "</td>";
        echo "</tr>";
    }
    echo "</tbody>";
    echo "</table>";
} else {
    echo "Erreur lors de l'exécution de la requête : " .
mysqli_error($conn);
}
mysqli_close($conn);
?>
</body>
</html>
```

Quelques petits rappels à propos de ce code :

- On utilise `isset()` et `empty()` : `isset()` vérifie que le paramètre existe, `empty()` vérifie qu'il n'est pas vide (par exemple si l'utilisateur soumet le formulaire sans rien sélectionner).
- `mysqli_prepare()` : prépare la requête en remplaçant les valeurs variables par des ?
- `mysqli_stmt_bind_param()` : associe une variable au ?. Le second argument précise le type de la variable : `"s"` pour une chaîne de caractères (`string`), `"i"` pour un entier (`integer`), `"d"` pour un décimal (`double`)
- `mysqli_stmt_execute()` : exécute la requête
- `mysqli_stmt_get_result()` : récupère le résultat sous la même forme qu'un `mysqli_query()`

Ajoute un second champ dans le formulaire de `choix.html` : un champ numérique demandant l'âge de l'utilisateur.

Adapte la requête SQL de `foraine.php` pour n'afficher que les attractions accessibles à cet âge (c'est-à-dire dont l'âge minimum est inférieur ou égal à l'âge fourni).

Le script PHP doit gérer quatre cas distincts (et donc quatre requêtes SQL différentes !) :

1. L'utilisateur spécifie à la fois un type d'attraction et un âge
2. L'utilisateur spécifie uniquement un type d'attraction (absence de valeur dans le champs de l'âge)
3. L'utilisateur spécifie uniquement un âge (option « Toutes les attractions » sélectionnée)
4. L'utilisateur ne spécifie pas d'âge et sélectionne « Toutes les attractions »

Etape 6

Comment complèterais-tu le code HTML suivant avec du code PHP afin que la liste des types d'attraction proviennent directement de la base de données (et pas codée « en dur » comme c'est le cas actuellement) ?

```
        <select name="type" id="type">
            <option value="">Toutes</option>
            <option value="Manège">Manège</option>
            <option value="Montagne russe">Montagne russe</option>
            <option value="Sensations fortes">Sensations fortes</option>
            <option value="Attraction">Attraction</option>
        </select>
```

La réponse se trouve à la page suivante. Essaie d'abord de le faire toi-même avant d'aller voir !

Etape 7

Voici la réponse de la question précédente :

```
<select name="type" id="type">
    <option value="">Toutes</option>
    <?php
    $result = mysqli_query(
        $conn,
        "SELECT DISTINCT type FROM attractions ORDER BY type ASC"
    );
    if ($result) {
        while ($row = mysqli_fetch_assoc($result)) {
            echo "<option value='".$row["type"]."'>".
$row["type"]."</option>";
        }
    }
    ?>
</select>
```

L'avantage est immédiat : si tu ajoutes un nouveau type d'attraction dans la base de données (par exemple « Spectacle »), il apparaîtra automatiquement dans la liste déroulante sans avoir à modifier le code HTML. `SELECT DISTINCT` garantit qu'un même type n'apparaît qu'une seule fois dans la liste, même s'il est associé à plusieurs attractions.

En dessous du tableau, affiche les informations suivantes calculées en PHP à partir des données récupérées

- Le nombre d'attractions affichées
- Le prix moyen d'un ticket
- L'attraction la moins chère et la plus chère

💡 Pour cela, tu peux utiliser les fonctions PHP `mysqli_num_rows()` pour compter les résultats, et parcourir le tableau des résultats pour calculer le reste.

Le code final se trouve à la page suivante. Essaie d'abord de le faire toi-même avant d'aller voir !

Page `choix.php` :

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Fête foraine - Recherche</title>
</head>
<body>
    <form action="foraine.php" method="POST">
        <label for="type">Type d'attraction :</label>
<select name="type" id="type">
    <option value="">Toutes</option>
    <?php
$conn = mysqli_connect('localhost', 'root', '', 't');
if (!$conn) {
    die('Erreur de connexion : ' . mysqli_connect_error());
}
    $result = mysqli_query(
        $conn,
        "SELECT DISTINCT type FROM attractions ORDER BY type ASC"
    );
    if ($result) {
        while ($row = mysqli_fetch_assoc($result)) {
            echo "<option value='".$row["type"]."'>".$row["type"]."</option>";
        }
    }
    ?>
</select>
        <label for="age">Votre âge :</label>
        <input type="number" name="age" id="age" min="1" max="120">
        <input type="submit" value="Rechercher">
    </form>
</body>
</html>
```

Page foraine.php :

```php
<!DOCTYPE html>

<html>

<head>
    <title>Fête foraine</title>

</head>

<body>

<?php

$conn = mysqli_connect('localhost', 'root', '', 't');

if (!$conn) {

    die('Erreur de connexion : ' . mysqli_connect_error());

}

if (isset($_POST['type']) && !empty($_POST['type']) &&
isset($_POST['age'])&& $_POST['age'] !== '') {

    $stmt = mysqli_prepare($conn, "SELECT * FROM attractions WHERE type = ?
AND age_minimum <= ?");

    mysqli_stmt_bind_param($stmt, "si", $_POST['type'], $_POST['age']);

    mysqli_stmt_execute($stmt);

    $result = mysqli_stmt_get_result($stmt);

} elseif (isset($_POST['type']) && !empty($_POST['type'])) {

    $stmt = mysqli_prepare($conn, "SELECT * FROM attractions WHERE type
= ?");

    mysqli_stmt_bind_param($stmt, "s", $_POST['type']);

    mysqli_stmt_execute($stmt);

    $result = mysqli_stmt_get_result($stmt);

} elseif (isset($_POST['age']) && $_POST['age'] !== '') {

    $stmt = mysqli_prepare($conn, "SELECT * FROM attractions WHERE
age_minimum <= ?");

    mysqli_stmt_bind_param($stmt, "i", $_POST['age']);

    mysqli_stmt_execute($stmt);

    $result = mysqli_stmt_get_result($stmt);

} else {
```

```php
    $result = mysqli_query($conn, "SELECT * FROM attractions");
}
if ($result) {
    $attractions = [];
    while ($row = mysqli_fetch_assoc($result)) {
        $attractions[] = $row;
    }
    echo "<table>";
    echo "<thead>";
    echo "<tr>
            <th>Nom</th>
            <th>Type</th>
            <th>Âge minimum</th>
            <th>Prix (ticket)</th>
            <th>Places par tour</th>
          </tr>";
    echo "</thead>";
    echo "<tbody>";
    foreach ($attractions as $row) {
        echo "<tr>";
        echo "<td>" . $row['nom'] . "</td>";
        echo "<td>" . $row['type'] . "</td>";
        echo "<td>" . $row['age_minimum'] . " ans</td>";
        echo "<td>" . $row['prix_ticket'] . " €</td>";
        echo "<td>" . $row['places_par_tour'] . "</td>";
        echo "</tr>";
    }
    echo "</tbody>";
    echo "</table>";
```

```
    $nb = count($attractions);

    if ($nb > 0) {

        $totalPrix  = 0;

        $moinsChere = $attractions[0];

        $plusChere  = $attractions[0];


        foreach ($attractions as $row) {

            $totalPrix += $row['prix_ticket'];

            if ($row['prix_ticket'] < $moinsChere['prix_ticket'])

                $moinsChere = $row;

            if ($row['prix_ticket'] > $plusChere['prix_ticket'])

                $plusChere = $row;

        }

        echo "<p>Nombre d'attractions affichées : " . $nb . "</p>";

        echo "<p>Prix moyen d'un ticket : " . round($totalPrix / $nb, 2) . "
€</p>";

        echo "<p>Attraction la moins chère : " . $moinsChere['nom'] . " (" .
$moinsChere['prix_ticket'] . " €)</p>";

        echo "<p>Attraction la plus chère : " . $plusChere['nom'] . " (" .
$plusChere['prix_ticket'] . " €)</p>";

    } else {

        echo "<p>Aucune attraction ne correspond à ces critères.</p>";

    }

} else {

    echo "Erreur lors de l'exécution de la requête : " .
mysqli_error($conn);

}

mysqli_close($conn);

?>

</body>

</html>
```

Exercice 3 : Le festival de street food

L'objectif de cet exercice est de pratiquer l'insertion de données dans une base de données via un formulaire HTML.

Etape 1

Crée la base de donnée `streetfood` et crée la table suivante :

```sql
CREATE TABLE stands (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom_stand VARCHAR(100),
  cuisine VARCHAR(100),
  responsable VARCHAR(100),
  nb_employes INT,
  prix_moyen DECIMAL(5,2),
  vegetarien BOOLEAN
);
```

Etape 2

Crée un fichier `inscription.html` contenant un formulaire avec les champs suivants :

- Nom du stand (texte)
- Type de cuisine (texte)
- Nom du responsable (texte)
- Nombre d'employés (numérique, minimum 1)
- Prix moyen d'un plat en euros (numérique)
- Une case à cocher « Menu végétarien disponible »
- Un bouton de soumission « Inscrire mon stand »

Le formulaire envoie les données en POST vers `ajouter.php`.

Etape 3

Crée le fichier `ajouter.php` qui :

1. Récupère les données POST
2. Vérifie avec `isset()` que toutes les données nécessaires sont présentes
3. Exécute la requête INSERT suivante :

```php
<?php
$sql = "INSERT INTO stands (nom_stand, cuisine, responsable, nb_employes, prix_moyen, vegetarien) VALUES ('$nom_stand', '$cuisine', '$responsable', $nb_employes, $prix_moyen, $vegetarien)";
mysqli_query($conn, $sql);
?>
```

4. Affiche un message de confirmation : « Le stand [nom] a bien été enregistré ! »
5. Propose un lien pour revenir au formulaire et un lien vers la liste des stands

Etape 4

Crée un fichier `liste.php` qui affiche tous les stands inscrits dans un tableau HTML.

Dans la colonne « Végétarien », affiche ✅ ou ❌ selon la valeur stockée en base de données.

Etape 5

La version actuelle de `ajouter.php` est vulnérable aux injections SQL : un utilisateur malveillant pourrait entrer du code SQL dans les champs du formulaire et manipuler ta base de données.

Dans un vrai projet, on préférera les requêtes préparées (`mysqli_prepare()`). Change le script PHP pour qu'il soit robuste face aux injections.

Page `inscription.html` :

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Street food - Inscription</title>
</head>
<body>
    <h1>Inscrire mon stand</h1>
    <form action="ajouter.php" method="POST">
        <label for="nom_stand">Nom du stand :</label>
        <input type="text" name="nom_stand" id="nom_stand" required>
        <br>
        <label for="cuisine">Type de cuisine :</label>
        <input type="text" name="cuisine" id="cuisine" required>
        <br>
        <label for="responsable">Nom du responsable :</label>
        <input type="text" name="responsable" id="responsable" required>
        <br>
        <label for="nb_employes">Nombre d'employés :</label>
        <input type="number" name="nb_employes" id="nb_employes" min="1"
required>
        <br>
        <label for="prix_moyen">Prix moyen d'un plat (€) :</label>
        <input type="number" name="prix_moyen" id="prix_moyen" min="0"
step="0.01" required>
        <br>
        <label for="vegetarien">Menu végétarien disponible :</label>
        <input type="checkbox" name="vegetarien" id="vegetarien" value="1">
        <br><br>
        <input type="submit" value="Inscrire mon stand">
    </form>
</body>
```

```
</html>
```

Page `ajouter.php` :

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Street food - Confirmation</title>
</head>
<body>
<?php
$conn = mysqli_connect('localhost', 'root', '', 'streetfood');
if (!$conn) {
    die('Erreur de connexion : ' . mysqli_connect_error());
}
if (isset($_POST['nom_stand']) && isset($_POST['cuisine']) &&
isset($_POST['responsable']) && isset($_POST['nb_employes']) &&
isset($_POST['prix_moyen'])) {
    $vegetarien = isset($_POST['vegetarien']) ? 1 : 0;
    $stmt = mysqli_prepare($conn, "INSERT INTO stands (nom_stand, cuisine,
responsable, nb_employes, prix_moyen, vegetarien) VALUES
(?, ?, ?, ?, ?, ?)");
    mysqli_stmt_bind_param($stmt, "sssidi", $_POST['nom_stand'],
$_POST['cuisine'], $_POST['responsable'], $_POST['nb_employes'],
$_POST['prix_moyen'], $vegetarien);
    if (mysqli_stmt_execute($stmt)) {
        echo "<p>Le stand <strong>" . $_POST['nom_stand'] . "</strong> a
bien été enregistré !</p>";
    } else {
        echo "<p>Erreur lors de l'enregistrement : " . mysqli_error($conn) .
"</p>";
    }
    mysqli_stmt_close($stmt);
} else {
    echo "<p>Erreur : certaines données sont manquantes.</p>";
}
mysqli_close($conn);
?>
```

```
    <a href="inscription.html">Retour au formulaire</a>

    <a href="liste.php">Voir la liste des stands</a>

</body>

</html>
```

Note : on utilise l'opérateur ternaire ? : qui est l'équivalent de :

```
if (isset($_POST['vegetarien'])) {
    $vegetarien = 1;
} else {
    $vegetarien = 0;
}
```

Page `liste.php` :

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Street food - Liste des stands</title>
</head>
<body>
<h1>Stands inscrits</h1>
<?php
$conn = mysqli_connect('localhost', 'root', '', 'streetfood');
if (!$conn) {
    die('Erreur de connexion : ' . mysqli_connect_error());
}
$result = mysqli_query($conn, "SELECT * FROM stands");
if ($result) {
    echo "<table>";
    echo "<thead>";
    echo "<tr>
            <th>Nom du stand</th>
            <th>Cuisine</th>
            <th>Responsable</th>
            <th>Employés</th>
            <th>Prix moyen</th>
            <th>Végétarien</th>
          </tr>";
    echo "</thead>";
    echo "<tbody>";
    while ($row = mysqli_fetch_assoc($result)) {
        echo "<tr>";
        echo "<td>" . $row['nom_stand'] . "</td>";
        echo "<td>" . $row['cuisine'] . "</td>";
        echo "<td>" . $row['responsable'] . "</td>";
```

```
        echo "<td>" . $row['nb_employes'] . "</td>";
        echo "<td>" . $row['prix_moyen'] . " €</td>";
        echo "<td>" . ($row['vegetarien'] ? "✅" : "❌") . "</td>";
        echo "</tr>";
    }
    echo "</tbody>";
    echo "</table>";
} else {
    echo "<p>Erreur lors de la récupération des données : " .
mysqli_error($conn) . "</p>";
}
mysqli_close($conn);
?>
</body>
</html>
```

Exercice 4 : Le marché de Noël

L'objectif de cet exercice est de pratiquer la mise à jour et la suppression d'enregistrements.

Etape 1

Crée la base de donnée `marche_noel` et crée la table suivante :

```
CREATE TABLE chalets (
   id INT AUTO_INCREMENT PRIMARY KEY,
   nom_exposant VARCHAR(100),
   produit VARCHAR(100),
   emplacement VARCHAR(10),
   loyer_jour DECIMAL(6,2),
   actif BOOLEAN DEFAULT TRUE
);


INSERT INTO chalets VALUES
(NULL, 'Ateliers Piron', 'Décorations en bois', 'A1', 45.00, TRUE),
(NULL, 'Boucherie Lejeune', 'Boudin de Noël', 'B3', 60.00, TRUE),
(NULL, 'Épicerie fine Dubois', 'Foie gras et terrines', 'A2', 55.00, TRUE),
(NULL, 'Saveurs d\'Orient', 'Épices et thés', 'C1', 40.00, TRUE),
(NULL, 'Atelier Bougie', 'Bougies artisanales', 'B1', 40.00, TRUE);
```

Etape 2

Crée un fichier `gestion.php` qui affiche la liste de tous les chalets dans un tableau HTML. Pour chaque chalet, ajoute deux boutons dans une colonne « Actions » : Modifier et Supprimer.

Etape 3

Le bouton « Supprimer » doit rediriger vers `supprimer.php?id=X` (où X est l'identifiant du chalet). Ce fichier doit :

1. Récupérer l'identifiant via GET
2. Vérifier que cet identifiant existe bien en base de données avant de supprimer
3. Exécuter la requête `DELETE`
4. Rediriger automatiquement vers `gestion.php`

Etape 4

Le bouton « Modifier » redirige vers `modifier.php?id=X`. Cette page doit :

1. Récupérer les informations du chalet correspondant via une requête SELECT
2. Afficher un formulaire pré-rempli avec les valeurs actuelles
3. Proposer un bouton « Enregistrer les modifications »

Lorsque le formulaire est soumis (en POST), exécute une requête UPDATE pour mettre à jour les informations du chalet, puis redirige vers `gestion.php`.

Etape 5

Dans un vrai projet, supprimer définitivement un enregistrement est souvent une mauvaise pratique : on perd l'historique des données.

Modifie le comportement du bouton « Supprimer » pour qu'il ne supprime pas réellement le chalet, mais qu'il passe simplement le champ `actif` à `FALSE` (via une requête UPDATE).

Dans `gestion.php`, affiche les chalets inactifs en grisé et remplace leur bouton « Supprimer » par un bouton « Réactiver ».

Exercice 5 : Le parc d'attractions

L'objectif de cet exercice est de travailler avec plusieurs tables liées et d'exploiter les jointures SQL.

Etape 1

Crée la base de donnée `parc` et crée la table suivante :

```sql
CREATE TABLE employes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    prenom VARCHAR(100),
    fonction VARCHAR(100)
);
CREATE TABLE attractions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    zone VARCHAR(50),
    duree_minutes INT,
    id_responsable INT,
    FOREIGN KEY (id_responsable) REFERENCES employes(id)
);
CREATE TABLE visites (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom_visiteur VARCHAR(100),
    date_visite DATE,
    id_attraction INT,
    avis INT, -- note de 1 à 5
    FOREIGN KEY (id_attraction) REFERENCES attractions(id)
);
INSERT INTO employes VALUES
(NULL, 'Marchal', 'Julien', 'Technicien'),
(NULL, 'Fonteneau', 'Laura', 'Animatrice'),
(NULL, 'Pirard', 'Tom', 'Technicien');
```

```sql
INSERT INTO attractions VALUES
(NULL, 'Cyclone', 'Zone Adrénaline', 3, 1),
(NULL, 'Petit train', 'Zone Famille', 8, 2),
(NULL, 'Aqua Splash', 'Zone Eau', 5, 3),
(NULL, 'Maison hantée', 'Zone Frissons', 12, 1);
INSERT INTO visites VALUES
(NULL, 'Alice Dupont', '2024-07-14', 1, 5),
(NULL, 'Marc Renard', '2024-07-14', 2, 4),
(NULL, 'Julie Simon', '2024-07-15', 1, 4),
(NULL, 'Pierre Lecomte', '2024-07-15', 3, 3),
(NULL, 'Anna Bastin', '2024-07-16', 4, 5),
(NULL, 'Tom Libert', '2024-07-16', 1, 5);
```

Etape 2

Crée un fichier `attractions.php` qui affiche la liste des attractions avec, pour chaque attraction, le nom et le prénom du responsable (et non son identifiant). Pour cela, tu devras utiliser une jointure :

```sql
SELECT attractions.nom, attractions.zone, employes.nom AS nom_resp, employes.prenom AS prenom_resp
FROM attractions
JOIN employes ON attractions.id_responsable = employes.id
```

Affiche le résultat dans un tableau HTML mis en forme.

Etape 3

Crée un fichier `stats.php` qui affiche, pour chaque attraction, le nombre de visites et la note moyenne des avis. Utilise pour cela une jointure avec la table `visites` et les fonctions d'agrégation `SQL COUNT()` et `AVG()` :

```sql
SELECT attractions.nom, COUNT(visites.id) AS nb_visites, AVG(visites.avis) AS note_moyenne
FROM attractions
LEFT JOIN visites ON visites.id_attraction = attractions.id
GROUP BY attractions.id
```

ⓘ On utilise ici un `LEFT JOIN` plutôt qu'un `JOIN` : cela permet d'afficher également les attractions qui n'ont encore reçu aucune visite.

Affiche la note moyenne arrondie à une décimale (utilise la fonction PHP `round()`). Si aucune visite n'a été enregistrée pour une attraction, affiche « Aucun avis » à la place de la note.

Etape 4

Crée un fichier `fiche.php` accessible via l'URL `fiche.php?id=X`. Cette page doit afficher :

- Le nom et la zone de l'attraction
- Le nom complet du responsable et sa fonction
- La liste de toutes les visites enregistrées pour cette attraction (nom du visiteur, date, note sous forme d'étoiles ⭐)

Etape 5

Sur la page `fiche.php`, ajoute un petit formulaire permettant à un visiteur d'enregistrer son passage :

- Son nom (texte)
- La date de sa visite (champ date)
- Sa note (boutons radio de 1 à 5)

Les données sont envoyées en POST et insérées dans la table `visites`. La page se recharge ensuite automatiquement pour afficher la nouvelle visite dans la liste.

Etape 6 (dépassement)

Crée un fichier `dashboard.php` qui présente une vue d'ensemble du parc :

- L'attraction la mieux notée (toutes visites confondues)
- L'attraction la plus visitée
- Le responsable ayant le plus d'attractions sous sa charge
- Le nombre total de visites enregistrées dans la base

Toutes ces informations doivent être obtenues via des requêtes SQL (et non calculées en PHP).

Références

Les présents exercices ont été élaborés à l'aide des ressources suivantes :

- Forbes, A. (2013). The Joy of PHP: A Beginner's Guide to Programming Interactive Web Sites.
- Vaswani, V. (2021). PHP A Beginner's Guide.
- Cours de « Développement d'applications WEB » (2018), Hénallux